

Tema 4

Normalización

1. [Problemas del esquema relacional](#)
2. [Formas normales](#)
3. [Primera forma normal \(1FN\)](#)
4. [Dependencias funcionales](#)
5. [Segunda forma normal \(2FN\)](#)
6. [Tercera forma normal \(3FN\)](#)
7. [Forma normal de Boyce-Codd \(FNBC ó BCFN\)](#)
8. [Fases en el diseño de una base de datos](#)

1. Problemas del esquema relacional

Una vez obtenido el esquema relacional (diseño lógico) resultante del diagrama entidad/relación (diseño conceptual) que representa la base de datos, normalmente tendremos una buena base de datos. Pero otras veces, debido a fallos en el diseño o a problemas indetectables, tendremos un esquema que puede producir una base de datos que incorpore estos problemas:

- **Redundancia.** Se llama así a los datos que se repiten continua e innecesariamente por las tablas de las bases de datos. Es el primer síntoma de problemas y se detecta fácilmente. Cuando es excesiva, es evidente que el diseño hay que revisarlo.
- **Ambigüedades.** Datos que no clarifican suficientemente el registro al que representan. Los datos de cada registro podrían referirse a más de un registro o incluso puede ser imposible saber a qué ejemplar exactamente se están refiriendo. Es un problema muy grave y difícil de detectar.
- **Pérdida de restricciones de integridad.** Normalmente debido a dependencias funcionales. Más adelante se explica en detalle este problema. Se arreglan fácilmente siguiendo una serie de pasos concretos.
- **Anomalías en operaciones de modificación de datos**, como puede ser el hecho de que al insertar un solo elemento haya que repetir tuplas en una tabla para variar unos pocos datos, o que al eliminar un elemento suponga eliminar varias tuplas obligatoriamente (por ejemplo, que eliminar un cliente suponga borrar seis o siete filas de la tabla de clientes, sería un error muy grave y por lo tanto un diseño terrible).

El principio fundamental para realizar un buen diseño reside en que las tablas deben referirse a objetos o situaciones muy concretas, relacionados exactamente con elementos reconocibles por el sistema de información de forma inequívoca. Cada fila de una tabla debe representar inequívocamente un elemento reconocible en el sistema. Lo que ocurre es que conceptualmente es difícil agrupar esos elementos correctamente.

En cualquier caso, la mayor parte de problemas se agravan si no se sigue un modelo conceptual y se decide crear directamente el esquema relacional. En ese caso el diseño tiene una garantía casi asegurada de funcionar mal.

Cuando aparecen los problemas enumerados, se pueden resolver usando reglas de normalización. La normalización es un conjunto de reglas o técnicas que se aplican a las tablas relacionales para evitar redundancias, obteniendo así una base de datos optimizada y reduciendo el espacio de almacenamiento. Estas reglas pretenden eliminar relaciones no deseables entre los datos de una tabla, es decir, dependencias.

Estas reglas suelen forzar la división de una tabla en dos o más tablas para arreglar un determinado problema.

2. Formas normales

Las formas normales forman parte de la teoría de normalización iniciada por el propio Codd y continuada por otros autores (entre los que destacan Boyce y Fagin). Codd definió en 1970 la primera forma normal, y luego aparecieron la segunda, tercera, la Boyce-Codd, la cuarta y la quinta formas normales.

La teoría de formas normales es una teoría absolutamente matemática, pero en el presente manual se describen de forma más intuitiva.

Una tabla puede encontrarse en primera forma normal y no en segunda forma normal, pero no al contrario. Es decir, los números altos de formas normales son más restrictivos, con lo que la quinta forma normal cumple todas las anteriores.

Hay que tener en cuenta que muchos diseñadores opinan que basta con llegar a la forma Boyce-Codd. La cuarta, y sobre todo, la quinta forma normal tienen menos defensores. Hay quien opina que hay bases de datos peores en quinta forma normal que en tercera, por lo que dicen que hasta el tercer nivel de normalización es suficiente.

3. Primera forma normal (1FN)

Es una forma normal inherente al esquema relacional. Es decir, toda tabla realmente relacional la cumple.

Se dice que una tabla se encuentra en primera forma normal si ningún atributo de una tupla puede tomar más de un valor.

La tabla:

TRABAJADOR		
DNI	Nombre	Departamento
12121212A	Andrés	Mantenimiento
12345345G	Andrea	Dirección Gestión

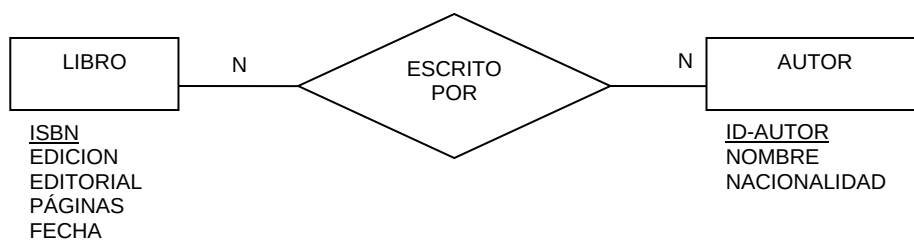
Visualmente es una tabla, pero no una tabla relacional (lo que en terminología de bases de datos relacionales se llama relación). Como hay atributos (Departamento) que tienen más de un valor, no cumple la primera forma normal. Estaría en primera forma normal si los datos fueran:

TRABAJADOR		
DNI	Nombre	Departamento
12121212A	Andrés	Mantenimiento
12345345G	Andrea	Dirección
12345345G	Andrea	Gestión

Ejemplo de aplicación de la 1ª Forma normal: Eliminación de valores múltiples en los atributos.

Supongamos que tenemos una entidad LIBRO con un atributo AUTOR, que puede contener varios autores. Como la 1FN nos obliga a que en un atributo de una tupla no pueda haber más que un valor, este diseño se transformará en una relación

entre 2 entidades, como se puede observar en la siguiente figura, para cumplir esta 1FN:



LIBRO (ISBN, EDICION, EDITORIAL, PÁGINAS, FECHA)
 AUTOR (ID-AUTOR, NOMBRE, NACIONALIDAD)
 ESCRITO_POR (ISBN↑, ID_AUTOR↑)

Otro ejemplo de aplicación de la 1ª Forma normal: Eliminación de valores múltiples en los atributos.

La tabla PEDIDOS:

NUM-PEDIDO	DIA-PEDIDO	LINEAS-PEDIDO	
Ord 1	6 junio 2018	NUM-PROD	CANT-PEDIDA
		P1	10
		P6	30
Ord 2	3 mayo 2018	NUM-PROD	CANT-PEDIDA
		P5	10
		P6	50
		P2	30

Los atributos NUM-PROD y CANT-PEDIDA tienen valores múltiples. La tabla relacional no está en 1ª forma normal.

Para normalizar la tabla se haría:

NUM-PEDIDO	DIA-PEDIDO	NUM-PROD	CANT-PEDIDA
Ord 1	6 junio 2018	P1	10
Ord 1	6 junio 2018	P6	30
Ord 2	3 mayo 2018	P5	10
Ord 2	3 mayo 2018	P6	50
Ord 2	3 mayo 2018	P2	30

4. Dependencias funcionales

Es un concepto fundamental en normalización.

Dependencia funcional

Se dice que un conjunto de atributos (Y) depende funcionalmente de otro conjunto de atributos (X) si para cada valor de X hay un único valor posible para Y.

Simbólicamente se denota por $X \rightarrow Y$.

Por ejemplo, el *nombre* de una persona depende funcionalmente del *DNI*; es decir para un *DNI* concreto sólo hay un *nombre* posible. En la tabla TRABAJADOR del ejemplo anterior (apartado 3), el *departamento* no tiene dependencia funcional, ya

que para un mismo *DNI* puede haber más de un *departamento* posible. Pero el *nombre* sí que depende del *DNI*.

Al conjunto *X* del que depende funcionalmente el conjunto *Y* se le llama **determinante**. Al conjunto *Y* se le llama **implicado**.

Dependencia funcional completa

Un conjunto de atributos (*Y*) tiene una dependencia funcional completa (depende completamente) de otro conjunto de atributos (*X*) si *Y* tiene dependencia funcional de *X* y además no se puede obtener de *X* un conjunto de atributos más pequeño que consiga una dependencia funcional de *Y* (es decir, no hay en *X* un determinante formado por atributos más pequeños).

Una dependencia funcional completa se denota como $X \Rightarrow Y$

Por ejemplo, en una tabla de *CLIENTES*, el conjunto de atributos formado por el *nombre* y el *dni* producen una dependencia funcional sobre el atributo *apellidos*; pero no es completa, ya que el *dni* individualmente, también produce una dependencia funcional sobre *apellidos*. El *dni* sí produce una dependencia funcional completa sobre el campo *apellidos*.

Dependencia funcional elemental

Se produce cuando $X \Rightarrow Y$ (hay una dependencia funcional completa) y además *Y* es un único atributo.

Dependencia funcional transitiva

Se puede producir cuando tenemos tres conjuntos de atributos *X*, *Y* y *Z*. Se da cuando *Y* depende funcionalmente de *X* ($X \rightarrow Y$), *Z* depende funcionalmente de *Y* ($Y \rightarrow Z$), pero además *X* no depende funcionalmente de *Y* (NO $Y \rightarrow X$). Entonces ocurre que *X* produce una dependencia funcional transitiva sobre *Z*. Es decir, si $X \rightarrow Y$ y $Y \rightarrow Z$, entonces $X \rightarrow Z$

Por ejemplo, supongamos que *X* es el atributo *Aula* de un instituto, e *Y* es el atributo *ZonaEvacuacion*. Entonces $X \rightarrow Y$ (la zona depende funcionalmente del aula, puesto que, dada un aula, sólo le corresponde una zona). Si *Z* representa el *NumPuerta*, entonces $Y \rightarrow Z$ (el número de puerta depende funcionalmente de la zona de evacuación, puesto que a cada zona le corresponde una sola puerta de evacuación). Como no ocurre que $Y \rightarrow X$ (el aula no depende funcionalmente de la zona ya que una zona se puede corresponder con varias aulas), entonces, $X \rightarrow Z$, es decir, el *NumPuerta* de salida depende transitivamente del *Aula* que ocupa la clase.

En otro ejemplo: *PRODUCTOS* (CódigoProducto, Nombre, Fabricante, País)

CódigoProducto $\rightarrow$ Fabricante

Fabricante $\rightarrow$ País

CódigoProducto - $\rightarrow$ País, es decir, CódigoProducto depende transitivamente de País.

5. Segunda forma normal (2FN)

Una tabla está en 2ª Forma Normal (2FN) **si está en primera forma normal y además, cada atributo que no sea clave depende de forma funcional**

completa respecto de la clave principal. Es decir, los atributos no clave dependen funcionalmente de la clave completa, no de parte de la clave.

Toda la clave principal debe hacer dependientes al resto de atributos; si hay atributos que dependen sólo de parte de la clave, entonces esa parte de la clave y esos atributos formarán otra tabla.

Con esto se pretende garantizar una correcta elección de claves y eliminar redundancias. Si las claves están formadas por un único atributo, entonces ese esquema estará seguro en 2FN.

Una vez alcanzado la 2FN, se controlan la mayoría de los problemas de lógica. Podemos insertar un registro sin un exceso de datos en la mayoría de las tablas.

Ejemplo de aplicación de la 2ª Forma normal: Eliminación de dependencias de parte de la clave

Sea una tabla de matriculaciones en la que cada alumno se puede matricular en varios cursos y en cada curso hay varios alumnos matriculados.

MATRICULACIONES

DNI	Cod Curso	Nombre	Apellido1	Nota
12121219A	34	Pedro	Valiente	9
12121219A	25	Pedro	Valiente	8
3457775G	34	Ana	Fernández	6
5674378J	25	Sara	Crespo	7
5674378J	34	Sara	Crespo	6

Suponiendo que el *DNI* y el *código de curso* formen la clave principal para esta tabla, sólo la *nota* tiene dependencia funcional completa. El *nombre* y los *apellidos* dependen de forma completa solo del *DNI*, con lo que la tabla no está en 2FN.

Para arreglarlo se construyen dos tablas, que sí estarán en 2FN: una con la clave y los atributos que depende de la clave completa (*DNI*, *CodCurso*, *Nota*) y otra con la parte de la clave y los atributos que dependen de ella (*DNI*, *Nombre*, *Apellido1*).

MATRICULACIONES

ALUMNOS		
DNI	Nombre	Apellido1
12121219A	Pedro	Valiente
3457775G	Ana	Fernández
5674378J	Sara	Crespo

DNI	Cod Curso	Nota
12121219A	34	9
12121219A	25	8
3457775G	34	6
5674378J	25	7
5674378J	34	6

Otro ejemplo de aplicación de la 2ª Forma normal: Eliminación de dependencias de parte de la clave

La tabla:

NUM-PEDIDO	DIA-PEDIDO	NUM-PROD	CANT-PEDIDA
Ord 1	6 junio 2000	P1	10
Ord 1	6 junio 2000	P6	30
Ord 2	3 mayo 2001	P5	10
Ord 2	3 mayo 2001	P6	50
Ord 2	3 mayo 2001	P2	30

cuya clave es NUM-PEDIDO, NUM-PROD está en 1FN, pero no está en 2FN porque el atributo DIA-PEDIDO que no es clave, depende funcionalmente sólo de NUM-PEDIDO, que es parte de la clave.

Una tabla que no está en 2FN puede descomponerse en otras dos tablas que sí estén en 2FN: una contendrá la parte de la clave de la que dependen otros atributos junto con esos atributos y la otra contendrá los demás atributos con la clave completa.

Por lo tanto, la tabla PEDIDOS se descompondrá en:

<u>NUM-PEDIDO</u>	DIA-PEDIDO
Ord 1	6 junio 2000
Ord 2	3 mayo 2001

Clave: NUM-PEDIDO

<u>NUM-PEDIDO</u>	<u>NUM-PROD</u>	CANT-PEDIDA
Ord 1	P1	10
Ord 1	P6	30
Ord 2	P5	10
Ord 2	P6	50
Ord 2	P2	30

Clave: NUM-PEDIDO, NUM-PROD

6. Tercera forma normal (3FN)

Ocurre **cuando una tabla está en segunda forma normal y además ningún atributo que no sea clave depende transitivamente de la clave de la tabla**. Es decir, hay que eliminar dependencias entre atributos no clave.

Para ello, **el atributo no clave Y que depende de otro no clave X se elimina de la tabla principal y ambos formarán una nueva tabla de la que X es clave**.

A los atributos que no pertenecen a la clave primaria se les puede denominar atributos no primos.

Cuando las tablas están en 3FN se previenen errores de lógica cuando se insertan o borran registros. Cada columna en una tabla está identificada de manera única por la clave primaria, y no debe haber datos repetidos. Esto proporciona un esquema limpio y elegante, con el que es fácil de trabajar y expandir.

Por ejemplo:

ALUMNOS				
<u>DNI</u>	Nombre	Apellido1	Cod Provincia	Provincia
12121349A	Salvador	Velasco	34	Palencia
12121219A	Pedro	Valiente	34	Palencia
3457775G	Ana	Fernández	47	Valladolid
5674378J	Sara	Crespo	47	Valladolid
3456858S	Marina	Serrat	08	Barcelona

La provincia depende funcionalmente del código de provincia, lo que hace que no esté en 3FN. Para convertirla a 3FN, el arreglo sería:

ALUMNOS			
DNI	Nombre	Apellido1	Cod Provincia
12121349A	Salvador	Velasco	34
12121219A	Pedro	Valiente	34
3457775G	Ana	Fernández	47
5674378J	Sara	Crespo	47
3456858S	Marina	Serrat	08

PROVINCIA	
Cod Provincia	Provincia
34	Palencia
47	Valladolid
08	Barcelona

Ejemplo de aplicación de la 3ª Forma normal: Eliminación de dependencias entre atributos no clave

Como se ha dicho, una tabla está en 3FN si está en 2FN y además no hay ninguna dependencia funcional entre atributos no clave. Esto hay veces que no ocurre. Por ejemplo, consideremos la tabla VEHÍCULOS que almacena información sobre coches.

Tabla: VEHICULOS Clave: MATRICULA

MATRICULA	PROPIETARIO	MODELO	NºCILINDROS
YX-01	Javier	Ford Laser	4
YJ-77	Alicia	Ford Falcon	6
YW-30	Javier	Toyota Corolla	4
YJ-37	Alicia	Ford Laser	4
YJ-83	Andrés	Toyota Corolla	4

Esta tabla está en 2FN ya que los atributos no clave dependen de la clave entera. Sin embargo, hay otras dependencias funcionales menos deseables: el atributo no clave NºCILINDROS depende de otro atributo no clave, como es el MODELO. Luego esta relación no está en 3FN.

La tabla puede descomponerse en otras tablas que estén en 3FN, metiendo los atributos dependientes en una tabla y los no dependientes en otra. Por lo tanto, la tabla VEHICULOS anterior se descompondría en dos: la de VEHICULOS, pero sin la columna de NUM_CILINDROS, y la de CILINDROS con clave MODELO.

Tabla: VEHICULOS Clave: MATRICULA

MATRICULA	PROPIETARIO	MODELO
YX-01	Javier	Ford Laser
YJ-77	Alicia	Ford Falcon
YW-30	Javier	Toyota Corolla
YJ-37	Alicia	Ford Laser
YJ-83	Andrés	Toyota Corolla

Tabla: CILINDROS Clave: MODELO

MODELO	NºCILINDROS
Ford Laser	4
Ford Falcon	6
Toyota Corolla	4

7. Forma normal de Boyce-Codd (FNBC ó BCFN)

Ocurre si una tabla está en tercera forma normal y además todo determinante (atributo del que dependen otros) es una clave candidata (no se puede repetir). Por ejemplo:

ORGANIZACIÓN		
Trabajador	Departamento	Responsable
Alex	Producción	Felipa
Arturo	Producción	Martín
Carlos	Ventas	Julio
Carlos	Producción	Felipa
Gabriela	Producción	Higinio
Luisa	Ventas	Eva
Luisa	Producción	Martín
Manuela	Ventas	Julio
Pedro	Ventas	Eva

Como se puede observar en la tabla anterior, un trabajador puede trabajar en varios departamentos. En cada departamento hay varios responsables, pero cada trabajador sólo tiene asignado uno.

El detalle importante que no se ha tenido en cuenta es que el responsable sólo puede ser responsable en un departamento. Este último detalle produce una dependencia funcional ya que: *Responsable*→*Departamento*

Por lo tanto, hemos encontrado un determinante que no es clave candidata, ocasionando que esta tabla no esté en FNBC.

En este caso la redundancia ocurre por una mala selección de la clave. La redundancia del departamento es completamente evitable.

La solución sería tener la tabla PERSONAL con la clave Trabajador+Responsable:

PERSONAL	
Trabajador	Responsable
Alex	Felipa
Arturo	Martín
Carlos	Julio
Carlos	Felipa
Gabriela	Higinio
Luisa	Eva
Luisa	Martín
Manuela	Julio
Pedro	Eva

Y la tabla RESPONSABLES, con clave Responsables:

RESPONSABLES	
Responsables	Departamento
Felipa	Producción
Martín	Producción
Julio	Ventas
Higinio	Producción
Eva	Ventas

Otro ejemplo de la aplicación de la FNBC:

Supongamos que en una academia los alumnos se apuntan a las clases de los distintos profesores. Un profesor puede tener varios alumnos. Cada curso lo da un profesor. Un profesor puede impartir varios cursos. Cada alumno puede matricularse en un curso por un determinado número de meses.

Se tiene la siguiente tabla:

Alumno	Profesor	Curso	Pagado	Meses
Inés Ortiz	Juan	Inglés 1	Sí	3
Luis Marín	Ana	Inglés 2	No	---
Luis Marín	Juan	Inglés 1	Sí	6
Lorenzo Arce	Pedro	Inglés 3	Sí	6
Lorenzo Arce	Juan	Francés 1	Sí	4

La clave de la tabla es Alumno+Profesor. Si intentamos aplicar las formas normales, nos damos cuenta de que cumple las 3 primeras: no hay más que un valor en cada atributo y tupla; no hay atributos que dependan sólo de Alumno o sólo del Profesor; y no hay dependencias entre atributos no clave (entre Curso, Pagado y Meses). Luego, está en 3FN. Sin embargo, hay una dependencia que hay que eliminar, ya que *Curso*--->*Profesor* y además el atributo Curso puede repetirse, es decir, es un determinante que puede repetirse. Entonces, según la definición de la FNBC, la clave está mal elegida. Debería ser Alumno+Curso y la dependencia entre Curso y Profesor sacarla a otra tabla.

Alumno	Curso↑	Profesor	Pagado	Meses
Inés Ortiz	Inglés 1	Juan	Sí	3
Luis Marín	Inglés 2	Ana	No	2
Luis Marín	Inglés 1	Juan	Sí	6
Lorenzo Arce	Inglés 3	Pedro	Sí	6
Lorenzo Arce	Francés 1	Juan	Sí	4

Curso	Profesor
Inglés 1	Juan
Inglés 2	Ana
Inglés 3	Pedro

Nota final:

También existen las formas normales 4FN y 5FN, más restrictivas que las anteriores, pero que su estudio queda fuera del alcance de este curso.

8. Fases en el diseño de una base de datos

Sea cual sea el SGBD que se vaya a utilizar, se requerirá un análisis previo de los datos del problema que se vaya a resolver para poder ser implementado. Para ello, se suelen desarrollar las siguientes fases

- **Recolección y análisis de requerimientos:** los diseñadores entrevistan a los futuros usuarios de la base de datos para recoger y documentar sus necesidades de información. En paralelo, conviene definir los requerimientos funcionales, las operaciones (transacciones) que se aplicarán a la base de datos, incluyendo las de obtención y actualización de datos.
- **Diseño conceptual:** una vez recogidos todos los requerimientos, el siguiente paso es crear un esquema conceptual para la base de datos mediante un modelo de representación de datos de alto nivel. El esquema conceptual contiene una descripción detallada de los requerimientos de información de los usuarios y descripciones de los tipos de datos, relaciones entre ellos y restricciones. Nosotros utilizaremos para el diseño de esquemas conceptuales el modelo E-R (entidad-relación), que describe los datos como entidades, vínculos (relaciones) y atributos (propiedades).
- **Diseño lógico de la base de datos (transformación del modelo de datos):** el siguiente paso en el proceso de diseño consiste en obtener el modelo lógico, que depende del modelo de BD que se vaya a utilizar, transformando el modelo conceptual al modelo de datos empleados por el SGBD (jerárquico, red o relacional). En nuestro caso, será el relacional, el más utilizado por las empresas en la actualidad. Sobre el modelo relacional se harán las operaciones de normalización, que darán más coherencia a la base de datos diseñada.
- **Diseño físico de la base de datos:** en este paso se especifican las estructuras de almacenamiento internas y la organización de los archivos de la base de datos

